Megamind

Stream 1 Memory Engine: Filtering, Storage and Retrieval Plan

Devin · builds on design doc v0.1 and the screenmem benchmarks

v0.1 — Hackathon build

Abstract

This document turns Stream 1 of the Megamind design (passthrough keyframes → memory tokens → store) into an implementation plan that is *efficient* (100 days of wear in well under 100 GB, bounded VLM cost per hour) and *queryable* (exact text, semantic, visual, temporal and activity questions, all answered with a cited frame). It keeps every interface in `docs/api.md` and the `MemoryStore` abstraction, and lists the few additive contract changes it needs. Numbers marked MEASURED come from the screenmem prototype benchmarks (`docs/research/screenmem/`); numbers marked est are estimates to be confirmed with the feeder on hour 3.

Contents

1 Goals and Non-Goals **2**
- 1.1 Goals . . . 2
- 1.2 Non-goals . . . 2

2 Pipeline Overview **2**

3 Filtering **3**
- 3.1 Why the v0.1 filter is not enough . . . 3
- 3.2 Stages . . . 3
- 3.3 Enrichment budget . . . 3

4 Memory Records **4**
- 4.1 Memory token (additive changes) . . . 4
- 4.2 Memory episode (new table) . . . 4
- 4.3 Vector sizes . . . 4

5 Image Storage Tiers **5**
- 5.1 Storage budget . . . 5

6 Retrieval **5**
- 6.1 Store layout (LanceDB) . . . 5
- 6.2 Query pipeline . . . 6
- 6.3 Activity questions . . . 6

7 Contract Changes **6**

8 Evaluation **6**

9 Work Plan **7**

10 Risks **7**

1 Goals and Non-Goals

1.1 Goals

- **Efficient ingest:** keep only frames that add information; spend VLM calls only on those.
- **Efficient storage:** hot JPEG for recent frames, compact video for old frames, text and vectors forever. Target: ≤ 15 GB per 100 days at the default budget, ≤ 100 GB per 100 days even when keeping every sampled frame.
- **Queryable:** every question type in Table 1 reaches the right frame or episode in the top 5, and every answer cites a frame that can still be shown.
- **Contract-compatible:** `POST /v1/frames`, `GET /v1/memories`, `recall_memory` keep their shapes; additions are optional fields and one new endpoint.

1.2 Non-goals

- Persisting learned visual tokens. Measured cost ≈ 164 KB per frame (2k tokens, fp16) versus 4.5–12 KB for the compressed frame; the Qwen reader creates visual tokens at query time instead (design D2).
- Face recognition or identity claims; multi-user tenancy.

Question type	Example	Answered by
Object location	"Where did I leave my keys?"	caption + entities (BM25, text vector), image vector
Exact text	"What was the name on the whiteboard?"	verbatim OCR (BM25) + high-res crop to reader
Visual / appearance	"The poster with the red bike"	SigLIP text→image vector
Temporal	"What was on my desk at 3 pm?"	time filter + nearest token
Activity	"What did I do after lunch?"	episodes over a time range
Speech (optional)	"What did Sam say about the demo?"	transcript field (BM25, text vector)

Table 1: Question types the engine must serve.

2 Pipeline Overview

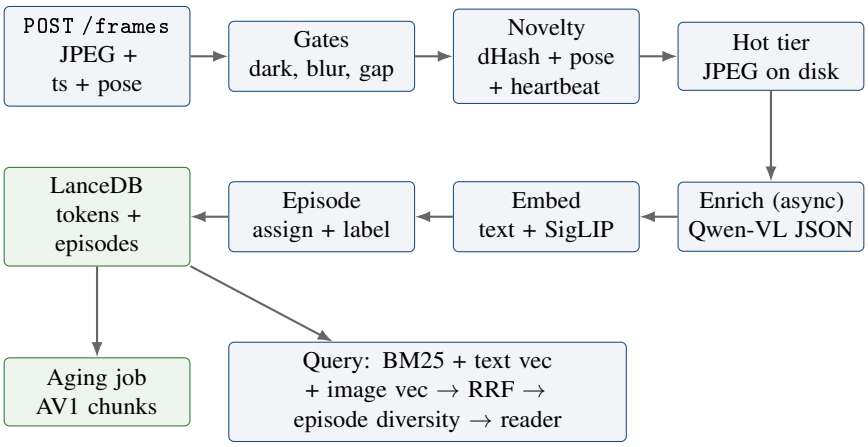


Figure 1: Stream 1 data flow. Blue: per-frame processing. Green: storage. The request path ends at the hot tier (202 returned); everything below runs asynchronously.

Decision

S1 — The request path stays cheap. `POST /frames` only decodes, gates, hashes and writes the JPEG (target < 15 ms). VLM enrichment, embeddings, episode assignment and aging run in the async worker, so headset upload never blocks on a model.

3 Filtering

3.1 Why the v0.1 filter is not enough

Measured

On real camera test footage (1080p, sampled at 1 fps), a pixel-difference change detector at a camera-tuned threshold kept **100%** of frames: sensor noise and head motion mean "nothing changed" almost never happens. On screens the same idea dropped 81% of frames, but at an aggressive threshold it lost 47 of 49 frames of a typing phase and made 2 of 5 questions unanswerable.

So the 10–30% keep rate in v0.1 §4.2 is unverified, and a pure dHash rule either keeps nearly everything (camera noise) or drops the frame that mattered. The plan: cheap gates, then a novelty score with event triggers, then a per-minute budget.

3.2 Stages

Stage	Rule	Cost	Reason code
Dark	mean luma < 20 (lens covered, headset off)	<1 ms	`dark`
Blur	variance of Laplacian on 160 px grey $< \tau_b$ (fast head turns)	$\sim$1 ms	`blurry` (new)
Min gap	< 2 s since last kept frame	0	`too_soon`
Novelty	dHash distance to the *last kept* frame and the episode representative; keep if ≥ 6	<1 ms	`duplicate`
Triggers	always keep on: pose change (yaw $> 25°$ or move > 0.75 m since last kept), heartbeat every 30 s, client `force=true`	0	—
Budget	token bucket, default 6 kept/min per device; when empty only triggers pass	0	`budget` (new)

Table 2: Filter stages, in order. Thresholds live in `Settings` and are tuned with the feeder.

Decision

S2 — Compare to the last kept frame, not every frame in a 30 s window. Comparing against all recent frames lets a slow pan drift forever without a keep; comparing to the last kept frame plus the current episode's representative bounds drift and is O(1).

Decision

S3 — Pose is a first-class trigger. The headset already sends pose; turning to face a new wall or walking into another room is the strongest cheap signal of new content and needs no pixels.

3.3 Enrichment budget

VLM calls are the dominant per-frame cost. The worker enriches a kept frame with Qwen-VL only if it is (a) the first frame of an episode, (b) has SigLIP cosine < 0.92 to the last enriched frame, or (c) is a

trigger frame. Other kept frames get an image vector and inherit the episode's caption for BM25, flagged `enriched=false`; the reader still sees their pixels at query time. At 6 kept/min this is $\leq$ 360 VLM calls per hour of wear[est].

4 Memory Records

4.1 Memory token (additive changes)

```
MemoryToken {                       # v0.1 fields unchanged
  ...                               # id, device_id, ts, caption, ocr_text, entities,
                                    # pose, text_vec, image_vec, thumb_path, frame_hash
  episode_id: str                   # now filled at ingest (was stretch)
  transcript: str | null            # optional ambient speech overlapping ts
  source: "quest" | "feeder" | "phone"
  trigger: "novel" | "pose" | "heartbeat" | "forced"
  sharpness: float                  # Laplacian variance, used for representative choice
  enriched: bool                    # false = caption inherited from episode
  tier: "hot" | "cold" | "text"     # where the pixels live
  chunk_id: str | null              # cold tier: AV1 chunk file
  chunk_offset: int | null          # frame index inside the chunk
}
```

4.2 Memory episode (new table)

```
MemoryEpisode {
  id, device_id, start_ts, end_ts, n_tokens
  title: str          # zero-shot SigLIP scene label, e.g. "a kitchen"
  summary: str        # optional one-line Qwen/Claude summary when the episode closes
  rep_token_id: str   # sharpest, most-text frame; its JPEG is kept forever
  image_vec: f32[768] # mean of member image vectors (normalised)
  text_vec: f32[d]    # embedding of title + summary + member entities
  pose_centroid: {x,y,z} | null
}
```

New episode when: time gap $>$ 120 s, or SigLIP cosine of the new frame to the episode representative $<$ 0.80, or pose moves $>$ 3 m from the centroid. Short detours rejoin one of the last 4 episodes if they match it again within the gap.

Measured

Image-embedding split threshold 0.80 was calibrated on real screen keyframes: within an episode min 0.83 / median 0.99, across different content 0.73–0.90. It must be re-checked on Quest footage.

4.3 Vector sizes

Vector	dim	fp32	int8
text, OpenAI `3-small`	1536	6.0 KB	1.5 KB
text, bge-m3 / voyage-3	1024	4.0 KB	1.0 KB
image (SigLIP-base)	768	3.0 KB	0.8 KB

Table 3: Per-token vector cost. Dimensions follow `EMBED_PROVIDER`; the LanceDB schema is created from the provider's `dim`.

5 Image Storage Tiers

Tier	Age	Format	Served as
Hot	0–72 h	JPEG as uploaded (960 px, q75)	file, `/v1/thumbs/{id}.jpg`
Cold	72 h – 100 d	one AV1 chunk per episode, 960 px, CRF 50, frame locator per token	decoded on demand, LRU cache, same URL
Text	$>$ 100 d	no pixels except episode representative (WebP)	representative frame

Table 4: Fidelity aging. Text, OCR, captions and vectors are never aged.

Measured

Per-frame size on 1080p camera-style test video encoded at 1 fps: x265 1280 CRF 30: 36–61 KB; AV1 1280 CRF 50: 6.1–7.6 KB; AV1 960 CRF 50: 4.4–4.5 KB. On real screen chunks AV1 at the same width as a good HEVC source came out *larger*, so the cold tier must also choose CRF $\geq$ 50, not only re-quantise.

Decision

S4 — Chunk per episode. Frames within an episode are similar, so inter-frame coding works; an episode is also the unit that is deleted, exported or shown. Kept frames are sparse in time (every $\geq$ 2 s), so expect up to 2$\times$ the consecutive-frame size[est]: $\sim$9 KB per frame.

5.1 Storage budget

	Default budget	30% keep	100% keep
Kept frames per 16 h day	5 760	17 280	57 600
Hot JPEG, 3 days (100 KB[est])	1.7 GB	5.2 GB	17 GB
Cold AV1, 100 days (9 KB[est])	5.2 GB	16 GB	52 GB
Vectors, 100 days, int8 (2.3 KB)	1.3 GB	4.0 GB	13 GB
Text + metadata, 100 days (1 KB)	0.6 GB	1.7 GB	5.8 GB
Total for 100 days	**$\approx$ 9 GB**	**$\approx$ 27 GB**	**$\approx$ 88 GB**

Table 5: Estimated storage for 100 days of 16 h wear (sampling 1 fps). Replace the [est] figures with feeder measurements on hour 3.

6 Retrieval

6.1 Store layout (LanceDB)

- `tokens`: all MemoryToken columns; vector columns `text_vec`, `image_vec`; FTS index on `caption`, `ocr_text`, `entities_text`, `transcript`; scalar index on `ts`, `device_id`, `episode_id`.
- `episodes`: MemoryEpisode; vector columns; FTS on `title`, `summary`.
- The existing `InMemoryStore` stays as the test double and gets the same fusion code.

6.2 Query pipeline

1. **Filter:** device and time range (Claude passes ISO bounds; a regex fallback parses "at 3 pm", "this morning", "20 minutes ago" for `GET /memories`).
2. **Retrieve** top 50 from each: BM25 (AND of terms, fall back to OR), text-vector cosine, SigLIP text→image cosine.
3. **Fuse** with weighted RRF: $s(t) = \sum_r w_r/(60+\text{rank}_r(t))$, $w_{\text{bm25}} = w_{\text{text}} = 1.0$, $w_{\text{image}} = 0.6$ (`visual` mode: 1.5).
4. **Diversify:** at most 3 tokens per episode in the top k.
5. **Expand:** attach the episode title and the ± 30 s neighbours' captions as context lines.
6. **Pack for the reader:** episode representatives first, then top tokens; for OCR questions send a high-resolution crop around the text instead of the downscaled frame.

Measured

On the screenmem 16-question gold set: BM25 hit@1 87.5%, hit@5 93.8%; dense text hit@1 68.8%, hit@5 100%; their union reached 100% hit@5. A glasses-style gold set with visual-only and speech questions reached 8/8 hit@1 with three-signal fusion. These are small sets.

6.3 Activity questions

`GET /v1/episodes?from=&to=&q=` returns episodes in time order, ranked by the summed fused score of their tokens when `q` is given. Claude gets a second tool, `list_episodes(time_from, time_to, query?)`, so "what did I do after lunch" is one call returning 5–20 compact lines instead of 24 frames.

7 Contract Changes

All additive; one PR updates `docs/api.md`, `models.py` and `Dto.cs` together, per the integration rule.

Change	Effect on headset
`reason` gains `blurry`, `budget`	none (status HUD can show it)
optional `meta.force`, `meta.source`	optional; `force` on push-to-talk start keeps the frame the user is looking at
`MemoryHit` gains `episode_id`, `episode_title`, `tier`	none
new `GET /v1/episodes`	optional timeline panel
new tool `list_episodes`	none (server-side)

8 Evaluation

- **Fixture:** 20–30 minute walkthrough video (keys on counter, whiteboard, poster, person talking) fed with `tools/feeder.py -realtime`; also the public test clips used in screenmem.
- **Gold set:** JSON of `{q, expect_substrings | expect_token_ids | expect_episodes, time?}`; `tools/eval.py` reports hit@1, hit@5, mean latency.
- **Efficiency report:** `GET /v1/stats`: keep rate per reason, VLM calls/hour, bytes per tier, projected GB per 100 days.
- **CI:** mock embedders and a synthetic image fixture test filter decisions, fusion order, episode splits and aging round-trip (encode → decode a frame by locator).

9 Work Plan

#	Task	h est	Done when
1	Filter v2: blur, last-kept novelty, pose + heartbeat triggers, budget	1.5	unit tests; feeder keep-rate printed
2	Real embedders: OpenAI/Voyage text, local SigLIP image + text tower	1.5	`EMBED_PROVIDER` switch works
3	Qwen-VL enricher (JSON prompt, Claude Haiku fallback) + enrichment budget	1.0	captions on feeder frames
4	LanceDBStore: schema, FTS, vector search, weighted RRF, diversity	2.5	same tests pass on both stores
5	Episodes: assign, scene label, representative, `/v1/episodes`, `list_episodes`	2.0	"what did I do" returns ordered episodes
6	Aging: AV1 chunk per episode, locator, on-demand decode, delete	2.0	round-trip test; `/thumbs` serves cold frames
7	Eval + stats: gold set runner, `/v1/stats`, fixture video	1.5	hit@5 and GB/100 d on dashboard
	Total	**12**	

Order: 1 → 2 → 4 gives a queryable vertical slice; 3, 5, 6, 7 follow. Tasks 6 and 7 can run in parallel with others.

10 Risks

Risk

Keep rate on real passthrough is unknown. Mitigation: the budget caps it regardless; feeder measurement on hour 3 sets thresholds.

Risk

VLM enrichment latency or cost. Mitigation: enrichment budget (§3.3), queue drop-oldest, image-vector-only tokens stay searchable visually.

Risk

SigLIP on CPU is slow. Mitigation: batch in the worker; base model at 224 px is ∼50–100 ms per image on a laptop CPU est; GPU if available.

Risk

Bystander privacy. Mitigation: no identity prompts, local storage, purge by device and by time range, capture indicator on the HUD.